

Westbank Project Passwords - Chat Transcript and Study Guide

This PDF captures the substantive conversation about adding the Westbank project-password page to the Radix CMS, configuring MSSQL through appsettings.json, and understanding the resulting code.

Conversation transcript

User: I wanted to add a database table to a page without modifying the CMS code, and asked where custom application code could be placed.

Assistant: A root-level custom-code folder was added as the application extension boundary. Its index.js module is loaded during startup and can register routes using the existing application instance and database services.

User: I asked to merge the changes with 01_Working_Files.

Assistant: The custom-code branch was committed and merged into the main 01_Working_Files worktree. Existing local changes were committed first, merge conflicts were resolved while preserving both sets of changes, and the merged build passed.

User: I wanted to add the Westbank database table to views/pages/westbank-project-passwords.ejs. The database was MSSQL, database Intranet_blue, table DB_WestbankProjects, with Account, Password, ProjectName, and ID. Managers and Administrators should be able to view, edit, delete, and create entries.

Assistant: MSSQL support was added using the mssql package. A separate connection is read from root-level appsettings.json, while the CMS MySQL .env configuration remains unchanged. The page includes role checks, a table listing, New Account form, edit forms, delete confirmation, and parameterized MSSQL queries.

User: The page showed HTML instead of rendering as a web page.

Assistant: The custom renderer was updated to explicitly set response.type("html"), and the build and syntax checks passed.

User: I wanted to study what the code was doing.

Assistant: A walkthrough explained route registration, MSSQL pooling, access control, EJS rendering, database reads, create/update/delete actions, startup loading, and a recommended study order.

Files involved

custom-code/index.js: Application routes, MSSQL connection, authorization, validation, and CRUD queries.

custom-code/README.md: Instructions for custom application code and MSSQL settings.

views/pages/westbank-project-passwords.ejs: The table, New Account form, edit forms, and delete buttons.

src/utils/customCode.ts: Loads custom-code/index.js during startup.

src/app.ts: Calls loadCustomCode before the CMS public catch-all route.

views/themes/blue-20260916/index.ejs: Wraps page content in the site layout.

appsettings.example.json: Safe MSSQL configuration template. The real appsettings.json is ignored by Git because it contains credentials.

How the request flows

1: The server starts in src/app.ts.

- 2: loadCustomCode loads custom-code/index.js and calls register({ app, db }).
- 3: custom-code/index.js registers GET and POST routes for /westbank-project-passwords.
- 4: requireManagerOrAdministrator checks the logged-in session role.
- 5: The GET route obtains the MSSQL pool and selects rows from DB_WestbankProjects ordered by Account.
- 6: EJS renders the rows and forms.
- 7: The content is inserted into the blue-20260916 theme and returned as text/html.

Important code concepts

Routes: GET displays the page. POST routes create, edit, and delete records.

Middleware: Authorization runs before the route handler and can redirect, reject, or continue.

Sessions: The current user and role are read from request.session.user.

Connection pooling: The MSSQL connection is created on first use and reused later.

Parameterized SQL: The .input() calls bind values instead of concatenating user input into SQL.

EJS: <% %> executes code, <%= %> outputs escaped values, and <%- %> outputs trusted HTML such as rendered page content.

Validation: Account numbers are checked as non-negative integers; password and project name are required.

Key code examples

MSSQL query

```
SELECT ID, ProjectName, Account, Password
FROM DB_WestbankProjects
ORDER BY Account
```

Create validation

```
const account = getAccountNumber(request.body.account);
const password = getRequiredText(request.body.password, 'Password');
const projectName = getRequiredText(request.body.projectName, 'Project Name');
```

Parameterized update

```
await pool.request()
  .input('id', sql.Int, id)
  .input('projectName', sql.NVarChar(255), projectName)
  .input('account', sql.Int, account)
  .input('password', sql.NVarChar(255), password)
  .query('UPDATE DB_WestbankProjects SET ProjectName = @projectName, Account = @account, Password = @password WHERE ID = @id');
```

Recommended study order

- 1: Read custom-code/index.js from top to bottom.
- 2: Read views/pages/westbank-project-passwords.ejs and match each form action to its POST route.
- 3: Read src/utis/customCode.ts to understand module loading.
- 4: Read src/app.ts to see startup order and middleware.
- 5: Read views/themes/blue-20260916/index.ejs to see how content enters the layout.